M4010104 / M4020104: PROGRAMMING IN PYTHON · DIGITAL UNIVERSITY KERALA · CAPSTONE PROJECT SPECIFICATION

Project 17: Soil Macro-Nutrient (NPK) & Micro-Climate Crop Suitability Recommender

Effort Budget: 10–12 Hours | Core Stack: Pure Python, NumPy, Pandas, Matplotlib, Seaborn, Plotly | Max Marks: 25 | Instructor: Dr. Sunil TT

1. Executive Overview & Domain Context

Precision agriculture matches farm-level soil chemical composition—Nitrogen (N), Phosphorus (P), Potassium (K), and soil pH—alongside local atmospheric micro-climate (temperature, relative humidity, and rainfall) to the optimal agricultural crop variety. Empirical crop recommendation prevents fertilizer over-application and optimizes crop yield.

Your team will develop an interpretable agronomic recommendation engine in pure Python that validates soil chemistry and meteorological bounds across 2,200 farm samples, computes 22 crop centroid profiles in NumPy, evaluates a vectorized Z-score standardized Nearest-Centroid classifier, and models agro-ecological clusters answering the 12 core questions below.

Primary Public Dataset & Telemetry/Data Schema

The project utilizes the verified public Precision Agriculture Crop Recommendation Dataset on Kaggle (**`https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset`**). The dataset structure captures:

- **Raw Feature Attributes:** **`N`** (kg/ha), **`P`** (kg/ha), **`K`** (kg/ha), **`temperature`** (°C), **`humidity`** (% RH), **`ph`** (0–14), **`rainfall`** (mm), **`label`** (22 crops).
- **Engineered / Target Variables:** **`npk_ratio`**, **`soil_fertility_index`**, **`z_scaled_features`**, **`predicted_crop`**, **`centroid_distance`**.

2. Complete Project Directory & File Architecture

Each team must organize their repository into a standardized, production-grade layout. All application source code must reside inside a dedicated package (**`src/`**), with separate directories for raw/curated data, interactive development notebooks, visual figures, and formal documentation:

```
crop-suitability-recommender/
├── README.md                          # Repository documentation & GitHub Pages source
├── requirements.txt                   # Dependency specification (numpy, pandas, matplotlib, seaborn, p
├── run_pipeline.py                    # Standalone production CLI application entry point
├── data/
│   ├── raw/
│   │   └── crop_recommendation_raw.csv            # Raw uncleaned input dataset (including simulated/in
│   └── processed/
│       ├── clean_crop_soil.csv          # Curated, validated clean CSV dataset
│       └── crop_soil_audit.log           # Timestamped audit report detailing filtered/rejected records
├── src/
│   ├── __init__.py                    # Python package initialization
│   ├── ingestion.py                   # Defensive line-by-line file reader, range auditor & CSV exporte
│   ├── analyzer.py                    # CropRecommenderEngine OOP class, NumPy vectorization & Pandas a
│   └── visualizer.py                  # Matplotlib envelope plots, Seaborn heatmaps & Plotly HTML expor
├── notebooks/
│   └── development_cycle.ipynb        # Development cycle notebook: EDA, trials, tests & unit assertion
└── reports/
    ├── figures/
    │   ├── primary_envelope.png       # Matplotlib confidence envelope & regression overlay
    │   ├── correlation_heatmap.png    # Seaborn annotated feature correlation heatmap
    │   └── distribution_regimes.png   # Seaborn distribution across categorical/domain tiers
```

```
├── interactive_dashboard.html     # Standalone interactive Plotly HTML dashboard with controls
└── Project_Report.pdf             # 3-4 page final technical report
```

3. Dual Development Deliverables: Notebook vs. Standalone Program

To mirror industry software engineering standards, each student group must deliver two distinct, complementary implementations of their solution:

Deliverable A: The Development Cycle Notebook (`notebooks/development_cycle.ipynb`)

The development notebook chronicles your team's engineering journey from raw data exploration to validated model prototypes. It is not merely a final script pasted into cells, but an authentic record of the development cycle:

- **Phase 1: Exploratory Data Ingestion & Diagnostics:** Loading raw data, inspecting missing values, testing boundary rules, and logging rejected records.
- **Phase 2: Mathematical Formulation & Feature Verification:** Developing and validating NumPy array calculations against target benchmarks using `np.allclose()`.
- **Phase 3: OOP Class Architecture & Unit Tests:** Instantiating the `CropRecommenderEngine` class, executing automated unit assertions (e.g., verifying that passing invalid parameters raises an explicit `ValueError`), and testing edge cases.
- **Phase 4: Statistical Modeling & Regimen Exploration:** Implementing ordinary least-squares regression, rolling-window calculations, and multi-metric Pandas grouping.
- **Phase 5: Visual Prototype Development:** Iteratively designing and refining Matplotlib and Seaborn figures, followed by configuring the interactive Plotly HTML dashboard.

Deliverable B: The Final Standalone Python Program (`run_pipeline.py` & `src/`)

While the notebook documents the exploratory development cycle, your repository must also provide a production-ready, fully executable Python application. The program must run end-to-end directly from the terminal without requiring Jupyter:

- **Command-Line Interface (CLI):** Executed via `python run_pipeline.py --input data/raw/crop_recommendation_raw.csv --output-dir data/processed --report-dir reports`.
- **Modular Package Import:** The driver script imports cleanly separated domain logic from `src.ingestion`, `src.analyzer`, and `src.visualizer`.
- **Terminal Feedback & Exit Codes:** The program must print structured progress indicators, log clean/faulty record counts to the console, and return an exit code of `0` upon successful pipeline completion.
- **Automated Artifact Generation:** In a single execution pass, the script reads raw inputs, writes `data/processed/clean_crop_soil.csv`, saves `data/processed/crop_soil_audit.log`, exports static figures to `reports/figures/`, and generates the standalone `reports/interactive_dashboard.html`.

4. Technical Investigation Requirements & Research Questions (Q1–Q12)

Your team's implementation across the development notebook and standalone program must rigorously answer the following 12 course-aligned engineering questions:

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
File I/O & Exceptions (`src/ingestion.py`)	**Q1**	**Soil Chemistry & Micro-Climate Range Audit:** Parse raw agricultural CSV rows using try-except. Validate that N, P, K, and rainfall are non-negative, pH lies within [0, 14], temperature is realistic, and log errors.	open(), try-except (ValueError, FileNotFoundError), range checks

Course Topic & Script	Q#	Concrete Engineering & Analytical Task	Python Libraries & Methods
Data Curation & Export (`src/ingestion.py`)	**Q2**	**Curated Soil-Crop Dataset Export & Audit Log:** Write clean records to clean_crop_soil.csv while logging corrupted labels, extreme negative nutrient entries, and non-numeric fields in crop_soil_audit.log.	Defensive file writing, dictionary parsing, formatted log reporting
OOP Design (`src/ingestion.py`)	**Q3**	**Crop Recommender Engine Class:** Design an encapsulated class CropRecommenderEngine with methods load_from_csv(), train_nearest_centroid(distance_metric='euclidean'), and summary_report(). Ensure invalid metric raises ValueError.	OOP class, __init__, encapsulation, raise ValueError
NumPy Arrays (`src/analyzer.py`)	**Q4**	**Vectorized Crop Centroids & Scaling Verification:** Using only raw NumPy arrays, compute the 7-dimensional mean feature centroid for all 22 crop classes and apply Z-score scaling. Verify centroid matrix via np.allclose().	np.mean(axis=0), np.std(), array broadcasting, np.allclose()
NumPy Masking (`src/analyzer.py`)	**Q5**	**High-Water vs. Drought-Resistant Crop Masking:** Flag crop classes requiring high rainfall (> 200 mm, e.g., rice, coconut) in NumPy. Compare against drought-tolerant pulses using boolean masks to verify separation.	np.mean(), np.std(), boolean masks (&, |), Python set
NumPy Regression (`src/analyzer.py`)	**Q6**	**Rainfall vs. Crop Yield Proxy OLS Model:** Fit an ordinary least-squares model relating soil Nitrogen requirement to temperature and humidity across cereals via np.linalg.lstsq. Quantify regression residuals.	np.column_stack(), np.linalg.lstsq(), matrix product (@), R2
Pandas Series (`src/analyzer.py`)	**Q7**	**Crop Sample Balance & Feature Variance Audit:** Quantify sample counts per crop class (100 samples each) in Pandas. Compute variance and interquartile range (IQR) for N, P, K across plantation vs. pulse crops.	.groupby('label').agg(), .describe()
Pandas Groupby (`src/analyzer.py`)	**Q8**	**Soil pH-Rainfall Discretization & Pivot Table:** Discretize soil pH into Acidic (< 6.0), Neutral (6.0-7.5), and Alkaline (> 7.5); discretize rainfall into Low, Medium, High. Build a 3×3 pivot_table(margins=True) showing dominant crops.	pd.cut(), .groupby().agg(), .pivot_table(margins=True)
Pandas Reshaping (`src/analyzer.py`)	**Q9**	**Wide-to-Long Nutrient Profile Ranking via Melt:** Reshape N, P, K nutrient requirements into tidy long format via pd.melt() and extract top 3 crops with the highest Nitrogen, Phosphorus, and Potassium requirements.	pd.melt(), .groupby('Nutrient'), .nlargest(3)
Matplotlib (`src/visualizer.py`)	**Q10**	**Matplotlib 22-Crop NPK Centroid Bar Chart:** Plot mean Nitrogen, Phosphorus, and Potassium requirements across key Kerala crops (rice, coconut, coffee, banana, blackgram) as grouped bars in Matplotlib.	plt.subplots(), ax.bar(), custom legends, xticks rotation
Seaborn (`src/visualizer.py`)	**Q11**	**Seaborn NPK-Rainfall Pairplot & Soil Heatmap:** Render a Seaborn pairplot of N, P, K, and rainfall colored by top crop categories. Generate an annotated correlation heatmap between soil chemistry and climate.	df.corr(), sns.heatmap(), sns.pairplot(), sns.despine()
Plotly HTML (`src/visualizer.py`)	**Q12**	**Interactive 3D Soil-Climate Crop Recommender:** Build an interactive HTML dashboard (interactive_dashboard.html) with a 3D scatter plot (N vs. P vs. Rainfall) colored by crop with test soil input sliders.	plotly.express.scatter_3d(), custom hover tooltips, write_html()

5. Collaborative Git Workflow, GitHub Pages & README.md Specification

Best Practices: Collaborative Git Workflow & GitHub Pages Deployment from Day 1

From the very first day, the student team must adopt professional version control and documentation practices. On Day 1, one designated team member initializes the central public GitHub repository, establishes a clear branch protection rule preventing direct pushes to **`main`**, adds all teammates as registered **Contributors** (with write access), and grants read access to the Course Instructor (**`Dr. Sunil TT`**) and teaching assistants. Immediately thereafter, the team must activate **GitHub Pages** (via *Settings → Pages → Build and deployment: Deploy from a branch: main /docs or root*) so that project documentation, interactive architecture diagrams, and the live Plotly dashboard (**`reports/interactive_dashboard.html`**) are publicly accessible and continuously deployed throughout the development lifecycle. All feature development must follow a strict branching model (e.g., **`feature/ingestion`**, **`feature/analysis`**, **`feature/visualization`**): individual team members must write small, atomic, well-commented commits from their own personal GitHub accounts and merge them into **`main`** exclusively through formal **Pull Requests (PRs)** accompanied by peer review. Dumping entire codebases in a single, monolithic end-of-semester commit is strictly prohibited and will result in zero marks for individual Git contribution; maintaining an unbroken, transparent Git commit and PR audit trail is essential for passing the code ownership defense during the live viva-voce examination.

Repository README.md Specification Template

The root **`README.md`** serves as the primary technical landing page and must strictly conform to the following structural format:

````
# Project 17: Soil Macro-Nutrient (NPK) & Micro-Climate Crop Suitability Recommender
[![Course](https://img.shields.io/badge/Course-Programming%20in%20Python-blue.svg)](https://duk.ac.in)
[![Python](https://img.shields.io/badge/Python-3.11%2B-brightgreen.svg)](https://python.org)
[![License](https://img.shields.io/badge/License-MIT-yellow.svg)](LICENSE)

## 1. Project Overview & Domain Context
- **Institution:** Digital University Kerala (MTech AI / MSc AI)
- **Course:** M4010104 / M4020104 — Programming in Python | **Instructor:** Dr. Sunil TT
- **Dataset:** Precision Agriculture Crop Recommendation Dataset on Kaggle (https://www.kaggle.com/datas

## 2. System Architecture & Pipeline Design
[Provide an ASCII or Mermaid architecture flow diagram showing raw ingestion -> validation -> analyzer -

## 3. Installation & Environment Setup
```bash
git clone https://github.com/<your-team>/crop-suitability-recommender.git
cd crop-suitability-recommender
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt
```

## 4. Execution Instructions
### A. Running the Standalone Application
```bash
python run_pipeline.py --input data/raw/crop_recommendation_raw.csv --output-dir data/processed --report
```
### B. Running the Development Notebook
```bash
jupyter notebook notebooks/development_cycle.ipynb
```

## 5. Key Analytical & Statistical Findings (Q1–Q12 Summary)
[Provide a concise markdown summary table detailing raw vs. clean counts, regression coefficients, anoma

## 6. Interactive Telemetry Dashboard
Explore the live interactive Plotly visualization hosted on GitHub Pages:
🔗 [Live Project Dashboard](https://<your-team>.github.io/crop-suitability-recommender/interactive_dashb
````

```
## 7. Contributor Git Commit & Pull Request Log Table
| Contributor Name | GitHub Handle | Assigned Module / Tasks | Verified Commit Hash Range | Merged Pull
| :--- | :--- | :--- | :--- | :--- |
| [Student Full Name] | `@github_handle` | Modular Architecture (`src/ingestion.py`, `src/analyzer.py`,
```

6. Evaluation Rubric & Mandatory Academic Integrity Governance

Projects are evaluated on a 25-mark scale based on code quality, mathematical correctness, visualization depth, git collaboration discipline, and live technical defense:

Assessment Criterion	Basis of Evaluation & Deliverable Verification	Max Marks
1. Data Curation & File I/O Pipeline (Q1–Q2)	Robust line-by-line file reading with defensive **`try-except`** handling, physical/domain bounds validation, clean CSV output, and structured **`crop_soil_audit.log`** reporting.	**5 Marks**
2. OOP, NumPy & Pandas Analytics (Q3–Q9)	Modular **`CropRecommenderEngine`** class architecture, vectorized NumPy feature re-derivations (verified via **`np.allclose`**), regression modeling, and Pandas rolling/groupby/melt operations.	**6 Marks**
3. Multi-Library Visualizations (Q10–Q12)	Publication-quality Matplotlib confidence envelope/trend plot, Seaborn correlation heatmap and distribution violin plot, and standalone interactive Plotly HTML dashboard.	**5 Marks**
4. Software Packaging, GitHub & Documentation	Clean directory layout, comprehensive **`README.md`**, working GitHub Pages site, **`development_cycle.ipynb`** notebook, standalone **`run_pipeline.py`**, and balanced individual Git commits/PRs.	**4 Marks**
5. Live Presentation & Code Viva-Voce Defense	Technical walkthrough and individual live viva defense where each student must explain or modify lines of code in real time without external assistance.	**5 Marks**

STRICT ACADEMIC INTEGRITY & ZERO-TOLERANCE AI POLICY (0 MARKS PENALTY): All submitted Python source code (**`.py`**, **`.ipynb`**) and written documentation will be screened through an automated **AI-Code & Plagiarism Detection Suite**. If AI-generated code or synthetic documentation is detected at any checkpoint or final submission, **0 Marks will be awarded immediately** for the entire project. Each student must prove authentic individual code ownership during the live viva-voce examination — *Course Instructor: Dr. Sunil TT*.

7. Milestone Schedule, Submission Checklist & Viva Defense Prompts

To ensure systematic progress across the semester, teams must adhere to three project checkpoints and prepare for on-the-spot live code modification challenges during the viva-voce examination:

Milestone Stage	Scheduled Window	Required Deliverables & Verification Milestones
Checkpoint 1 (Data Ingestion)	Week 11 Lab Session	Repository initialized with contributors and GitHub Pages active. **`src/ingestion.py`** operational with line-by-line filtering, **`clean_crop_soil.csv`** exported, and **`crop_soil_audit.log`** generated.
Checkpoint 2 (Analytics & OOP)	Week 13 Lab Session	**`src/analyzer.py`** complete with **`CropRecommenderEngine`** class, NumPy feature checks (Q4), outlier masks (Q5), regression modeling (Q6), and exploratory notebook sections.
Final Submission & Viva	Week 15 Project Review	Full package operational: standalone **`run_pipeline.py`**, polished **`development_cycle.ipynb`**, interactive **`interactive_dashboard.html`** on GitHub Pages, 3–4 page PDF report, and live code defense.

Representative Live Viva-Voce Code Modification Challenge

During the individual code viva-voce, the instructor will ask students to execute an immediate 2-minute code modification on their working repository to demonstrate genuine authorship:

- **Challenge Prompt:** *"In src/analyzer.py, adjust the nearest-centroid distance metric to weight rainfall twice as heavily as soil NPK. Re-run your recommendation function on a sample soil profile and report the shift in top-3 recommended crops."*